



Bachelorarbeit

Systematische Transformation KI-generierter
Web-Architekturen in sichere, lokale
Desktop-Anwendungen unter Anwendung des
Local-First-Paradigmas

Joshua Ohans

Prüfer: Prof. Dr. Matthias Teßmann
Prof. Dr. Ronald Petrlc

© Joshua Ohans, 2026

Diese Arbeit, einschließlich ihrer Teile, **ist urheberrechtlich geschützt**. Jede Verwertung außerhalb der engen Grenzen des Urheberrechtsgesetzes ohne Zustimmung von dem Urheber ist untersagt und strafbar. Dies gilt insbesondere für Vervielfältigungen, Übersetzungen, Mikroverfilmung und die Speicherung und Verarbeitung in elektronischen Systemen.

Danksagung

Only if you want to thank someone.

Kurzdarstellung

Kurze Zusammenfassung der Arbeit, höchstens halbe Seite. Deutsche Fassung auch nötig, wenn die Arbeit auf Englisch angefertigt wird.

Schlagwörter

KI-Prototyp, Local-First, Desktop-Anwendung, Sicherheit, Elektron, Transformation

Inhaltsverzeichnis

1	Einleitung	5
2	Theoretischer Hintergrund	6
2.1	KI-generierte Software: Werkzeuge, Qualität und Grenzen	6
2.1.1	Überblick über KI-gestützte Entwicklungstools	6
2.1.2	Charakteristika und Qualitätsdefizite KI-generierten Codes	7
3	Methodisches Vorgehen	9
4	Analyse und Problemanalyse	10
5	Entwurf und Entwicklung des Artefakts	11
6	Evaluation und Demonstration	12
7	Diskussion	13
8	Fazit und Ausblick	14
A	Supplemental Information	20

Abkürzungsverzeichnis

BaaS Backend-as-a-Service

CWE Common Weakness Enumeration

IDE Integrated Development Environment

LLM Large Language Model

Kapitel 1

Einleitung

Kapitel 2

Theoretischer Hintergrund

2.1 KI-generierte Software: Werkzeuge, Qualität und Grenzen

Die vorliegende Arbeit setzt bei einem KI-generierten Web-Prototyp an, dessen Entstehung und Charakteristika für das gesamte Transformationsvorhaben grundlegend sind. Dieser Abschnitt ordnet zunächst die Landschaft der KI-gestützten Entwicklungswerkzeuge ein, aus der ein solcher Prototyp typischerweise hervorgeht. Anschließend werden die spezifischen Qualitätsdefizite herausgearbeitet, die den mit diesen Werkzeugen erzeugten Code auszeichnen, und deren Konsequenzen für eine sichere Weiterverwendung erörtert.

2.1.1 Überblick über KI-gestützte Entwicklungstools

Seit der Einführung leistungsfähiger Large Language Models (LLMs) hat sich eine Praxis etabliert, bei der Software überwiegend durch natürlichsprachliche Prompts anstelle manuell geschriebenen Codes entsteht. Der von Andrej Karpathy geprägte Begriff *Vibe Coding* beschreibt diese Praxis treffend als ein Vorgehen, bei dem Entwicklerinnen und Entwickler den generierten Code akzeptieren, ohne ihn im Detail zu prüfen, und sich stattdessen auf das beobachtbare Verhalten der Anwendung verlassen [1]. Diese Arbeitsweise senkt die Einstiegshürde für die Softwareentwicklung erheblich, da auch technische Laien funktionsfähige Prototypen erstellen können [2]. Aus dieser Entwicklung ist eine heterogene Landschaft spezialisierter Werkzeuge entstanden, die sich hinsichtlich Zielsetzung, bevorzugtem Technologie-Stack und Grad der Cloud-Abhängigkeit deutlich unterscheiden.

Replit positioniert sich als Online-Integrated Development Environment (IDE) mit integriertem KI-Agenten und fokussiert sich auf vollständige Full-Stack-Web-Prototypen, deren Persistenzschicht standardmäßig an eine cloudseitige Datenbank (Replit DB bzw. PostgreSQL) gekoppelt ist und deren Ausführung ausschließlich in der Replit-Cloud erfolgt [3]. Diese Architekturentscheidung begünstigt zwar schnelle, sofort ausführbare Prototypen, erzeugt jedoch einen erheblichen Vendor-Lock-in und schließt eine native lokale Ausführung von vornherein aus. **Lovable** spezialisiert sich auf die Generierung vollständiger Anwendungen aus natürlicher Sprache und verknüpft den erzeugten Code standardmäßig mit einem Backend-as-a-Service (BaaS)-Anbieter, in der Regel Supabase, für Authentifizierung und Datenhaltung [4]. Dieses Kopplungsmuster beschleunigt die Prototyp-Erstellung, verlagert die Verantwortung für eine korrekte Sicherheitskonfiguration jedoch weitgehend auf die Endnutzenden, was sich in der Praxis wiederholt als Schwachstelle erwiesen hat, wie Unterabschnitt 2.1.2 anhand konkreter Vorfälle zeigt. **Cursor** und **GitHub Copilot** verfolgen demgegenüber einen assistenzbasierten Ansatz: Sie sind in einen bestehenden Entwickler-Workflow eingebettet und ergänzen dort die Codeerstellung zeilen- oder funktionsweise, stellen jedoch kein eigenständiges Prototyping-Werkzeug für vollständige Anwendungen dar. **Figma Make** adressiert wiederum ausschließlich die Übersetzung von Design-Artefakten in Frontend-Code und erzeugt pixelgenaue UI-Komponenten auf Basis

bestehender Figma-Designsysteme, ohne dabei Backend-Logik oder Geschäftsregeln zu berücksichtigen [5]. Weitere Werkzeuge wie Bolt.new und v0 von Vercel besetzen ähnliche Nischen zwischen reinem UI-Prototyping und vollständiger Full-Stack-Generierung, wobei auch sie überwiegend auf eine cloud-gebundene Deployment-Pipeline setzen.

Tabelle 2.1 fasst die zentralen Vergleichsdimensionen dieser Werkzeuglandschaft zusammen. Trotz der Unterschiede im Detail zeigt sich ein gemeinsames Muster: Nahezu alle Werkzeuge sind auf eine Cloud-native, stateless Web-Architektur mit React- oder Next.js-Frontend und einem verwalteten Backend-Dienst ausgerichtet. Für das in dieser Arbeit betrachtete Transformationsszenario ist dies insofern relevant, als der Ausgangszustand des zu migrierenden Prototyps durch genau diese architektonischen Grundannahmen geprägt ist, unabhängig davon, mit welchem konkreten Werkzeug er erzeugt wurde.

Werkzeug	Zielsetzung	Bevorzugter Stack	Architektur-Charakter
Replit	Full-Stack-Prototyping	React/Node.js, PostgreSQL	Cloud-gebunden, monolithisch
Lovable	Full-Stack-Prototyping	React/Next.js, Supabase	Cloud-gebunden, BaaS-gekoppelt
Cursor / Copilot	Assistenzbasierte Codegenerierung	sprachagnostisch	In bestehende Codebasis eingebettet
Figma Make	Design-zu-Code	React/Vue/HTML+CSS	Statisch, reines Frontend
Bolt.new / v0	UI- und App-Prototyping	React/Next.js	Cloud-gebunden, deploy-mentororientiert

Tabelle 2.1: Vergleich KI-gestützter Entwicklungswerkzeuge nach Zielsetzung, Technologie-Stack und Architektur-Charakter

2.1.2 Charakteristika und Qualitätsdefizite KI-generierten Codes

KI-generierter Code entsteht als Ergebnis eines Sprachmodells, das plausible Fortsetzungen auf Basis von Trainingsdaten produziert, nicht jedoch auf Basis einer bewussten architektonischen Planung. Daraus resultiert ein Black-Box-Charakter: Designentscheidungen, Abhängigkeitsstrukturen und Sicherheitsannahmen werden vom Modell nicht dokumentiert und sind aus dem Code selbst häufig nicht rekonstruierbar. Dieser Befund lässt sich anhand des Produktqualitätsmodells nach ISO/IEC 25010 systematisch einordnen, das Softwarequalität in Merkmale wie Wartbarkeit, Sicherheit, Kompatibilität und Zuverlässigkeit unterteilt [6]. Empirische Untersuchungen zeigen für mehrere dieser Merkmale konsistente Defizite bei KI-generiertem Code.

Hinsichtlich der **Sicherheit** liefert eine wachsende Zahl unabhängiger Studien ein weitgehend übereinstimmendes Bild. Pearce *et al.* fanden bei einer systematischen Analyse von 1.689 durch GitHub Copilot generierten Programmen, dass rund 40 % der Vorschläge in sicherheitsrelevanten Szenarien verwundbaren Code enthielten [7]. Tihanyi *et al.* bestätigten dieses Muster mit dem FormAI-v2-Datensatz, der 331.000 durch neun verschiedene LLMs erzeugte C-Programme umfasst: Mittels formaler Verifikation wurden bei mindestens 62,07 % der Programme Schwachstellen nachgewiesen, wobei sich die untersuchten Modelle in ihrer Fehleranfälligkeit kaum unterschieden [8]. Asare *et al.* relativierten dieses Bild insofern, als Copilot bei der Reproduktion bekannter Schwachstellenmuster aus menschlich verursachten Common Weakness Enumerations (CWEs) tendenziell seltener fehlerhaft reagierte als die ursprünglichen menschlichen Entwickler, was zeigt, dass die Befunde nicht als pauschale Unterlegenheit von KI-Code gegenüber menschlichem Code misszuverstehen sind [9]. Besonders relevant für die vorliegende Arbeit ist der Befund von Perry *et al.*, dass Entwicklerinnen und Entwickler mit Zugang zu einem KI-Assistenten nicht nur signifikant unsichereren Code produzierten, sondern zugleich überzeugter waren, sicheren Code geschrieben zu haben, als die Kontrollgruppe ohne KI-Unterstützung [10]. Dieses Auseinanderfallen von wahrgenommener und tatsächlicher Sicherheit ist für den in dieser Arbeit betrachteten Anwendungsfall bedeutsam, da Nutzende von

No-Code-Prototyping-Werkzeugen typischerweise über keine softwaretechnische Ausbildung verfügen und die Sicherheitsannahmen des generierten Codes daher kaum eigenständig hinterfragen können. Ein verwandtes, im Kontext von Lieferketten relevantes Risiko sind sogenannte *Package Hallucinations*: Spracklen *et al.* wiesen in einer Analyse von 576.000 Codebeispielen aus 16 verschiedenen LLMs nach, dass durchschnittlich mindestens 5,2 % der von kommerziellen und bis zu 21,7 % der von quelloffenen Modellen vorgeschlagenen Paketabhängigkeiten schlicht nicht existieren [11]. Derartige halluzinierte Abhängigkeiten eröffnen ein neuartiges Angriffsszenario, bei dem Angreifende gezielt Pakete unter den vom Modell wiederholt genannten, aber nicht existierenden Namen veröffentlichen.

Auch bezüglich der **Wartbarkeit** zeichnet sich ein konsistenter Negativtrend ab. Eine Langzeitanalyse von GitClear über rund 153 Millionen geänderte Codezeilen zwischen 2020 und 2023 zeigt, dass der Anteil an kurzfristig wieder verworfenem oder überarbeitetem Code (*Code Churn*) im Zuge der breiten Copilot-Adoption deutlich anstieg, während der Anteil an bewusst refaktoriertem Code parallel zurückging [12]. Dieser Befund deckt sich mit der in der Praxis beobachteten Tendenz KI-generierten Codes zu geringer Kopplungsdisziplin und mangelnder Wiederverwendung bestehender Strukturen, was der *Don't-Repeat-Yourself*-Maxime der Softwaretechnik zuwiderläuft. Strukturell äußert sich dies in einer fehlenden Trennung von Zuständigkeiten (*Separation of Concerns*), da generierter Code primär auf die Erfüllung einer im Prompt beschriebenen Funktion optimiert wird und nicht auf die langfristige Erweiterbarkeit der Architektur.

Nicht zuletzt relativiert eine kontrollierte Studie der Forschungsorganisation METR das verbreitete Narrativ pauschaler Produktivitätssteigerungen durch KI-Assistenz. In einem randomisierten Experiment mit 16 erfahrenen Open-Source-Entwicklerinnen und -Entwicklern, die reale Aufgaben in ihnen vertrauten, umfangreichen Repositories bearbeiteten, benötigten die Teilnehmenden bei erlaubter KI-Nutzung im Mittel 19 % mehr Zeit als ohne KI-Unterstützung, obwohl sie im Vorfeld eine Beschleunigung um 24 % erwartet hatten [13]. Dieser Befund ist nicht unmittelbar auf das No-Code-Prototyping durch technische Laien übertragbar, da die Studie erfahrene Entwickler in etablierten Codebasen mit hohen Qualitätsanforderungen untersuchte, doch er verdeutlicht, dass der Effekt von KI-Unterstützung auf Softwarequalität und -geschwindigkeit stark kontextabhängig und keineswegs durchgängig positiv ist.

Die Konsequenzen dieser Qualitätsdefizite lassen sich anhand dokumentierter Vorfälle konkretisieren. Im Fall von Lovable wurde bekannt, dass von 1.645 untersuchten, mit dem Werkzeug erstellten Webanwendungen 170 eine fehlerhafte Zugriffskontrolle in der angebundenen Supabase-Datenbank aufwiesen, wodurch persönliche Daten, Zahlungsinformationen und API-Schlüssel öffentlich zugänglich waren [14]. In einem weiteren dokumentierten Fall löschte der KI-Agent von Replit trotz expliziter, mehrfach wiederholter Anweisung eine Produktionsdatenbank und meldete der Nutzenden Partei anschließend fälschlicherweise, eine Wiederherstellung sei nicht möglich [15]. Beide Vorfälle illustrieren exemplarisch, dass die in den empirischen Studien nachgewiesenen Qualitäts- und Sicherheitsdefizite keine rein akademischen Randbefunde darstellen, sondern sich unmittelbar auf produktiv eingesetzte, KI-generierte Anwendungen auswirken.

Für die vorliegende Arbeit ergibt sich aus diesem Befund eine zentrale Implikation: KI-generierter Code kann nicht unverändert (*as-is*) in eine produktionsreife, sicherheitskritische Desktop-Anwendung überführt werden. Die in den folgenden Abschnitten dieses Kapitels behandelten Konzepte zur Web-zu-Desktop-Transformation, zum Local-First-Paradigma und zur Absicherung lokaler Anwendungen bilden daher nicht nur eine technische, sondern auch eine aus den hier dargestellten Qualitätsdefiziten begründete Notwendigkeit für ein systematisches Refactoring, das in Kapitel 3 methodisch hergeleitet und in den Kapiteln 4 und 5 auf die konkrete Fallstudie angewendet wird.

Kapitel 3

Methodisches Vorgehen

Kapitel 4

Analyse und Problemanalyse

Kapitel 5

Entwurf und Entwicklung des Artefakts

Kapitel 6

Evaluation und Demonstration

Kapitel 7

Diskussion

Kapitel 8

Fazit und Ausblick

Abbildungsverzeichnis

Quellcodeverzeichnis

Tabellenverzeichnis

2.1	Vergleich KI-gestützter Entwicklungswerkzeuge nach Zielsetzung, Technologie-Stack und Architektur-Charakter	7
-----	--	---

Literaturverzeichnis

- [1] Wikipedia contributors, „Vibe Coding,” Wikipedia, The Free Encyclopedia, 2026. [Online]. Available: https://en.wikipedia.org/wiki/Vibe_coding (Zitiert auf Seite 6.)
- [2] B. Edwards, „Will the Future of Software Development Run on Vibes?” *Ars Technica*, 2025. [Online]. Available: <https://arstechnica.com/ai/2025/03/is-vibe-coding-with-ai-gnarly-or-reckless-maybe-some-of-both/> (Zitiert auf Seite 6.)
- [3] Replit, „Replit Agent – Documentation,” 2026. [Online]. Available: <https://docs.replit.com/replitai/agent> (Zitiert auf Seite 6.)
- [4] Lovable, „Welcome to Lovable – Documentation,” 2026. [Online]. Available: <https://docs.lovable.dev/introduction/welcome> (Zitiert auf Seite 6.)
- [5] Figma, „Figma Make – Prompt to Code,” 2026. [Online]. Available: <https://www.figma.com/make/> (Zitiert auf Seite 7.)
- [6] ISO/IEC, „ISO/IEC 25010:2023 – Systems and Software Engineering – Systems and Software Quality Requirements and Evaluation (SQuaRE) – Product Quality Model,” International Organization for Standardization, 2023. [Online]. Available: <https://www.iso.org/standard/78176.html> (Zitiert auf Seite 7.)
- [7] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, und R. Karri, „Asleep at the Keyboard? Assessing the Security of GitHub Copilot’s Code Contributions,” in *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2022. [Online]. Available: <https://arxiv.org/abs/2108.09293> (Zitiert auf Seite 7.)
- [8] N. Tihanyi, T. Bisztray, M. A. Ferrag, R. Jain, und L. C. Cordeiro, „How Secure is AI-Generated Code: A Large-Scale Comparison of Large Language Models,” *Empirical Software Engineering*, 2024. [Online]. Available: <https://arxiv.org/abs/2404.18353> (Zitiert auf Seite 7.)
- [9] O. Asare, M. Nagappan, und N. Asokan, „Is GitHub’s Copilot as Bad as Humans at Introducing Vulnerabilities in Code?” *Empirical Software Engineering*, 2023. [Online]. Available: <https://arxiv.org/abs/2204.04741> (Zitiert auf Seite 7.)
- [10] N. Perry, M. Srivastava, D. Kumar, und D. Boneh, „Do Users Write More Insecure Code with AI Assistants?” in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2023, S. 2785–2799. [Online]. Available: <https://arxiv.org/abs/2211.03622> (Zitiert auf Seite 7.)
- [11] J. Spracklen, R. Wijewickrama, A. H. M. N. Sakib, A. Maiti, B. Viswanath, und M. Jadhwal, „We Have a Package for You! A Comprehensive Analysis of Package Hallucinations by Code Generating LLMs,” in *34th USENIX Security Symposium*, 2025. [Online]. Available: <https://arxiv.org/abs/2406.10279> (Zitiert auf Seite 8.)
- [12] GitClear, „Coding on Copilot: 2023 Data Suggests Downward Pressure on Code Quality,” GitClear, Tech. Rep., 2024. [Online]. Available: https://www.gitclear.com/coding_on_copilot_data_shows_ais_downward_pressure_on_code_quality (Zitiert auf Seite 8.)
- [13] J. Becker, N. Rush, E. Barnes, und D. Rein, „Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity,” METR, Tech. Rep., 2025. [Online]. Available: <https://metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study/> (Zitiert auf Seite 8.)
- [14] R. Albergotti, „The Hottest New Vibe Coding Startup May Be a Sitting Duck for Hackers,” *Semafor*, 2025. [Online]. Available: <https://www.semafor.com/article/05/29/2025/the-hottest-new-vibe-coding-startup-lovable-is-a-sitting-duck-for-hackers> (Zitiert auf Seite 8.)

- [15] S. Sharwood, „Vibe Coding Service Replit Deleted User’s Production Database, Faked Data, Told Fibs Galore,” The Register, 2025. [Online]. Available: https://www.theregister.com/2025/07/21/replit_saastr_vibe_coding_incident/ (Zitiert auf Seite 8.)

Anhang A

Supplemental Information

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc,

molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.